

Algorithms

14주차

Graph Algorithms

지난 주 수업 내용

- Graph
 - Vertices and Edges
 - Walk, Path, Cycle
 - Subgraph
 - Tree
 - Adjacency Lists and Adjacency Matrix
- Graph Algorithms
 - DFS, BFS, ...
- Minimum Spanning Tree
 - Kruskal's Algorithm

오늘의 수업 내용

- Graph Algorithms
 - Minimum Spanning Tree
 - Prim's Algorithm
 - Shortest Path Algorithms
 - Dijkstra's Algorithm
 - Floyd-Warshall Algorithm

Spanning Tree

- Let $G = (V, E)$ be a simple undirected graph

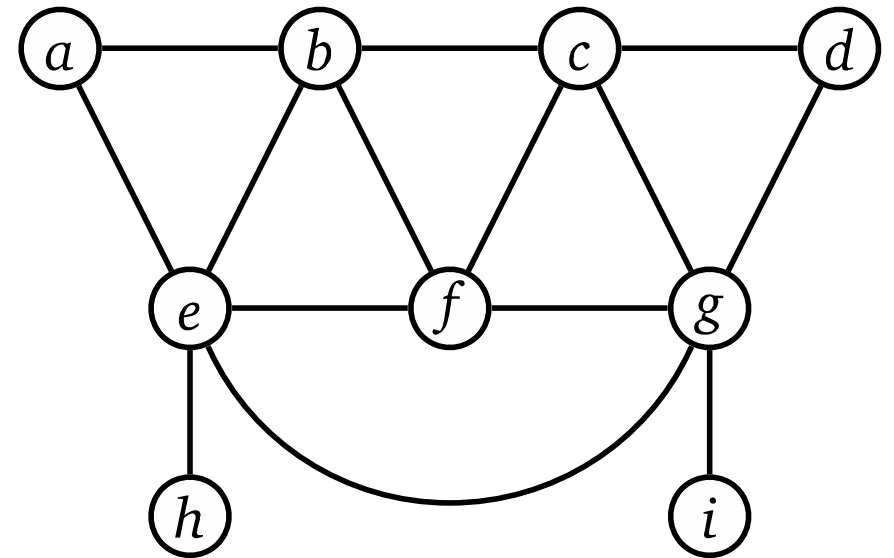
- Spanning Tree

- A **spanning tree of G** is a subgraph of G that is a tree and contains every vertex of G

- G 는 connected여야 spanning tree가 존재

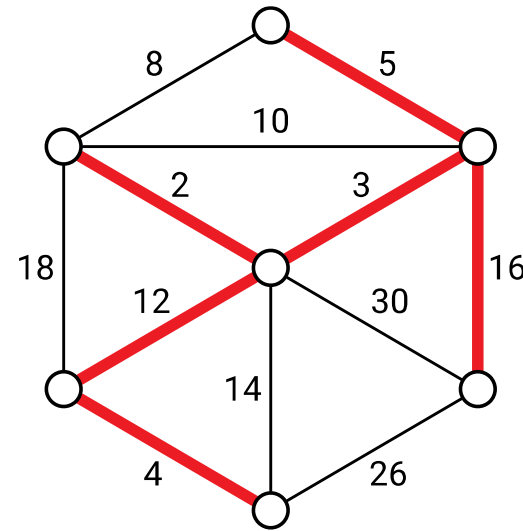
- Tree는 connected graph니까..

- Spanning tree is a connected subgraph with fewest edges



Minimum Spanning Tree

- Let $G = (V, E)$ be a **weighted** simple, undirected, connected graph
 - Weight: $w(u, v)$ for each edge $(u, v) \in E$
- Spanning Tree
 - A **spanning tree of G** is a subgraph of G that is a tree and contains every vertex of G
- Minimum(-weight) spanning tree
 - a spanning tree with smallest total weight
- MST는 연결성을 보장하는 weight가 최소인 subgraph



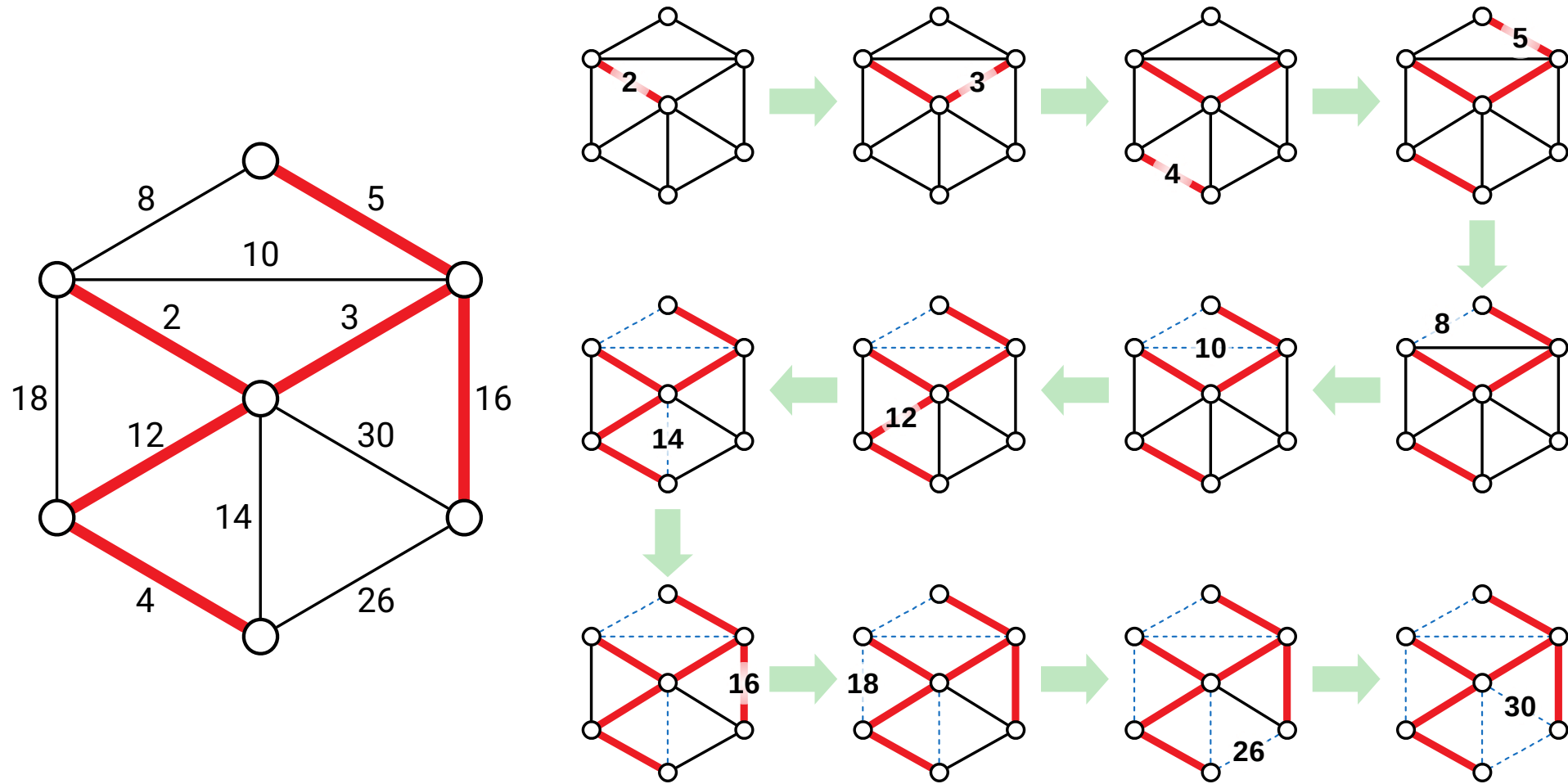
Kruskal's Algorithm

- Input: a weighted, simple, undirected, connected graph $G = (V, E)$
Output: a minimum spanning tree of G

- Kruskal's Algorithm

- Greedy algorithm
- 각 Edge를 최종 MST에 포함할까 말까의 결정의 연속으로 문제를 해결
- Greedy choice
 - weigh가 작은 edge를 우선 선택하여 추가
 - 단, 추가했을 때, cycle을 만들면 안됨

Kruskal's Algorithm



Kruskal's Algorithm

- Correctness Proof
 - By exchange argument
- Time complexity
 - $O(E \log V)$

Kruskal(V, E)

Sort E by increasing weight

Set $F \leftarrow (V, \emptyset)$

Initialize(V) // for disjoint sets

for $i = 1$ to $|E|$

Let $e = (u, v)$ be the i -th lightest edge in E

If **Find**(u) $\neq$ **Find**(v) // $F + e$ has no cycle

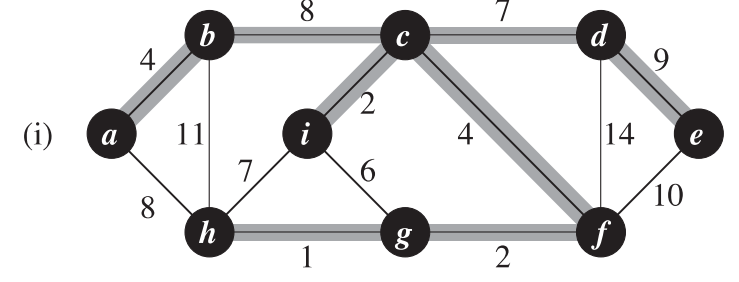
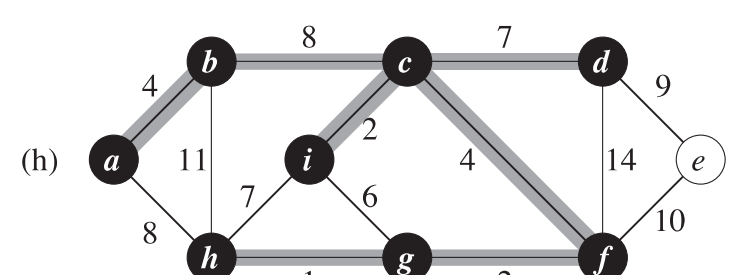
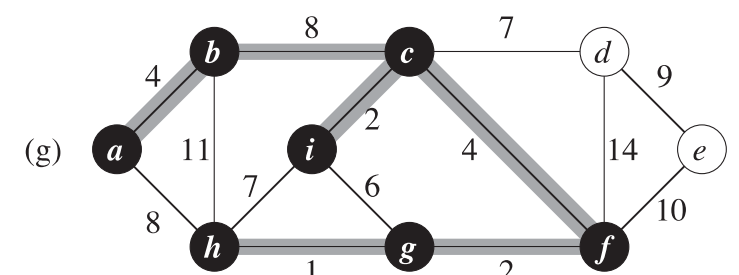
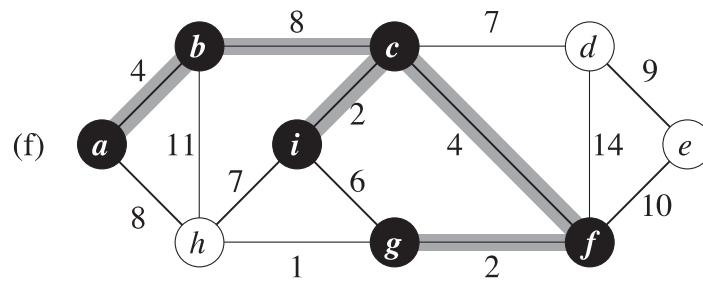
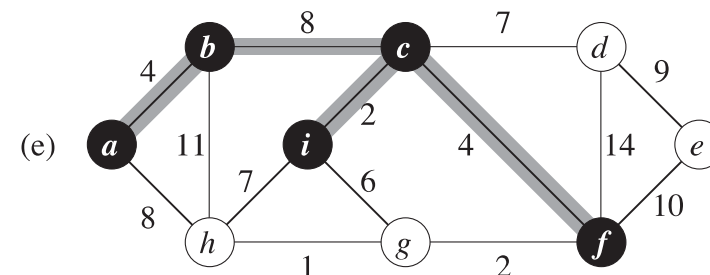
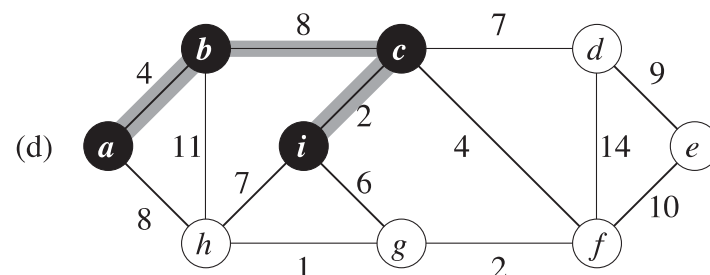
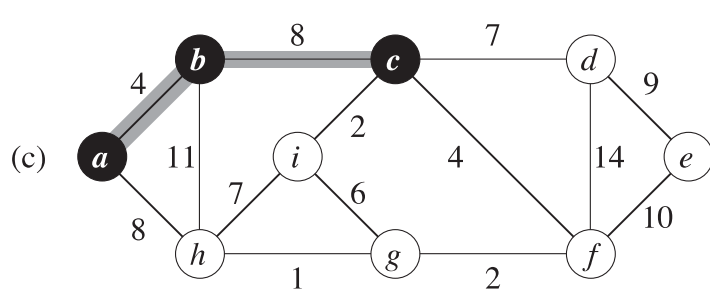
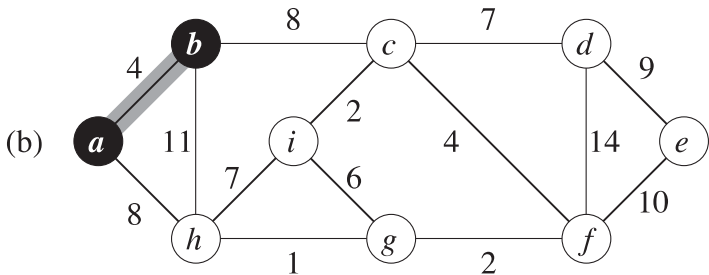
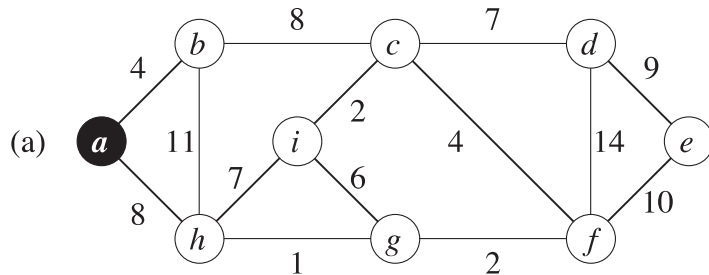
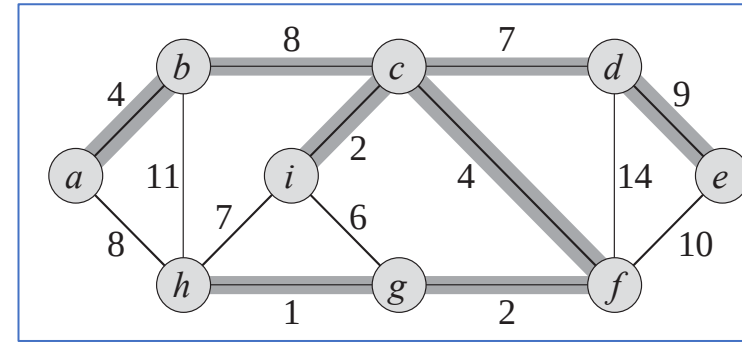
Add e to F and **Union**(u, v)

return F

Prim's Algorithm

- Input: a weighted, simple, undirected, connected graph $G = (V, E)$
Output: a minimum spanning tree of G
- Prim's Algorithm
 - Jarník's Algorithm으로도 불림 (Jarník이 1930년 최초로 발표, Prim은 1957)
 - Greedy Algorithm입니다
 - 임의의 vertex를 root로 시작하여 tree를 키워나가는 방식으로 MST를 계산
 - 매 시점에서 현재 tree에 하나의 vertex와 edge를 추가
 - 추가되는 vertex는 현재 tree에 포함되지 않은 것 중
“연결 비용이 가장 작은 것” → 즉 “weight가 가장 작은 edge와 연결된 것”
 - 이것이 greedy choice의 기준

Prim's Algorithm



Prim's Algorithm

- (실행 중간에) 필요한 정보 → 저장할 변수(자료구조)
 - 현재 tree의 형태 → 배열 T 로 표현
 - 현재 MST로 확정 포함되지 않은 vertex들 → 집합 Q
 - Q 에 있는 vertex에 대해 현재 tree에 연결하기 위한 비용(weight) → 배열 d
- 필요한 변수
 - Q : 집합으로 현재 tree에 포함되지 않은 vertex들을 저장한다
 - 최초에는 $Q = V$, 마지막에는 $Q = \emptyset$
 - d : 배열로 $d[v]$ 는 v 를 현재 확정 MST에 연결하기 위한 edge중 최소 weight로
 - 현재 확정 MST에서 v 로 가는 edge가 없는 경우에는 $d[v] = \infty$
 - T : 배열로 $T[v]$ 는 현재 tree에서 vertex v 의 부모를 저장한다
 - T 는 현재 확정 MST와는 약간 다르지만 최종적으로 minimum spanning tree가 됨
 - 현재 MST에 확정 포함된 vertex u 에 대해 $T[v] = u$ 라면 $d[v] = w(u, v)$
 - $d[v] = \infty$ 인 vertex v 에 대해서는 $T[v] = NULL$

Prim's Algorithm

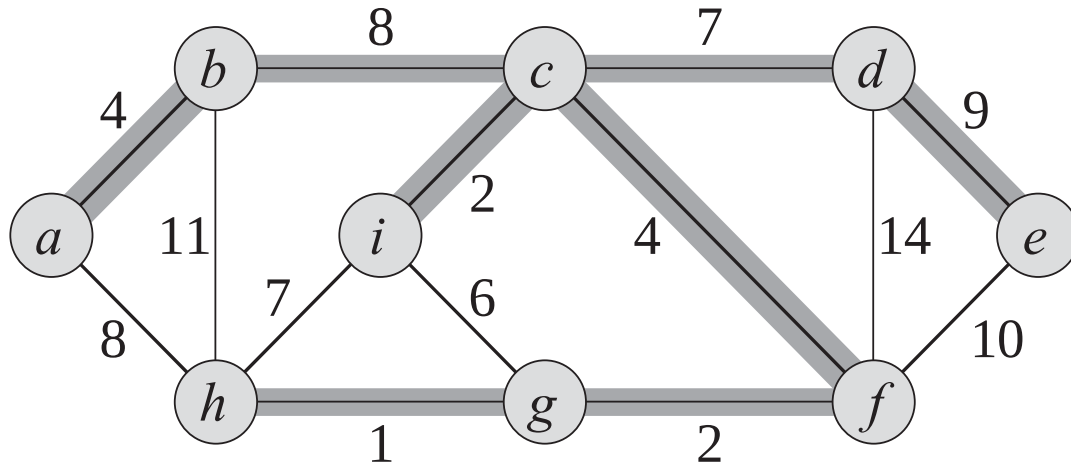
- 변수의 의미 (invariants)
 - Q : 현재 tree에 포함되지 않은 vertex들을 저장한다
 - d : $d[v]$ 는 v 를 현재 확정 MST에 연결하기 위한 edge중 최소 weight
 - T : $T[v]$ 는 현재 tree에서 vertex v 의 부모

```
Prim( $V, E, r$ )  // returns an MST with root  $r$ 
  for  $v \in V$ 
     $d[v] \leftarrow \infty$  and  $T[v] \leftarrow NULL$ 
   $Q \leftarrow V$ 
   $d[r] \leftarrow 0$   // root
  while  $Q \neq \emptyset$ 
    Let  $u \leftarrow$  the vertex in  $Q$  with smallest  $d$ -value
     $Q \leftarrow Q - \{u\}$ 
    for each  $v \in V$  such that  $(u, v) \in E$ 
      if  $d[v] > w(u, v)$ 
         $d[v] \leftarrow w(u, v)$  and  $T[v] \leftarrow u$   // update!
  return  $T$ 
```

Prim's Algorithm

$$Q = \{a, b, c, d, e, f, g, h, i\}$$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>
<i>d</i>	0	∞	∞	∞	∞	∞	∞	∞	∞
<i>T</i>									



```

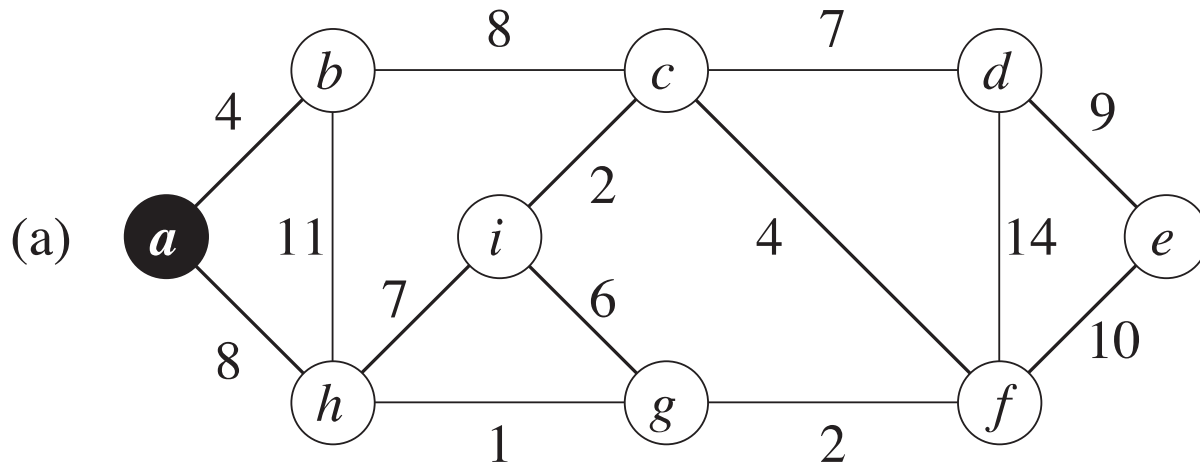
Prim(V, E, r)  // returns an MST with root r
for v ∈ V
    d[v] ← ∞ and T[v] ← NULL
Q ← V
d[r] ← 0  // root
while Q ≠ ∅
    Let u ← the vertex in Q with smallest d-value
    Q ← Q − {u}
    for each v ∈ V such that (u, v) ∈ E
        if d[v] > w(u, v)
            d[v] ← w(u, v) and T[v] ← u  // update!
return T
    
```

Q: 현재 tree에 포함되지 않은 vertex들
d[*v*]: *v*를 현재 MST에 연결하기 위한 cost
T[*v*]: 현재 tree에서 vertex *v*의 부모

Prim's Algorithm

$$Q = \{ b, c, d, e, f, g, h, i \}$$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>
<i>d</i>	0	4	∞	∞	∞	∞	∞	8	∞
<i>T</i>		<i>a</i>						<i>a</i>	



```

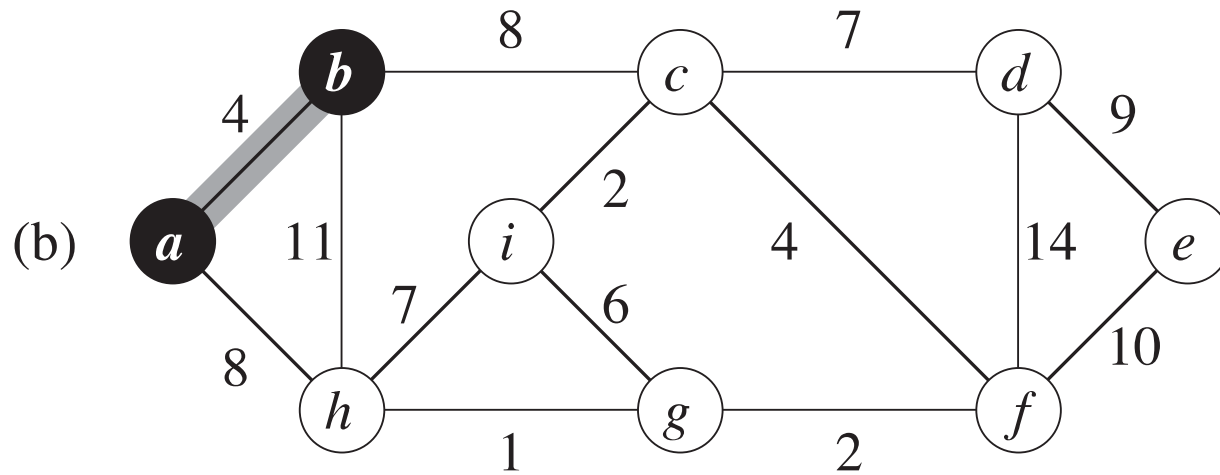
Prim(V, E, r)  // returns an MST with root r
for v ∈ V
    d[v] ← ∞ and T[v] ← NULL
Q ← V
d[r] ← 0  // root
while Q ≠ ∅
    Let u ← the vertex in Q with smallest d-value
    Q ← Q − {u}
    for each v ∈ V such that (u, v) ∈ E
        if d[v] > w(u, v)
            d[v] ← w(u, v) and T[v] ← u  // update!
return T
    
```

Q: 현재 tree에 포함되지 않은 vertex들
d[*v*]: *v*를 현재 MST에 연결하기 위한 cost
T[*v*]: 현재 tree에서 vertex *v*의 부모

Prim's Algorithm

$$Q = \{ c, d, e, f, g, h, i \}$$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>
<i>d</i>	0	4	8	∞	∞	∞	∞	8	∞
<i>T</i>		<i>a</i>	<i>b</i>					<i>a</i>	



```

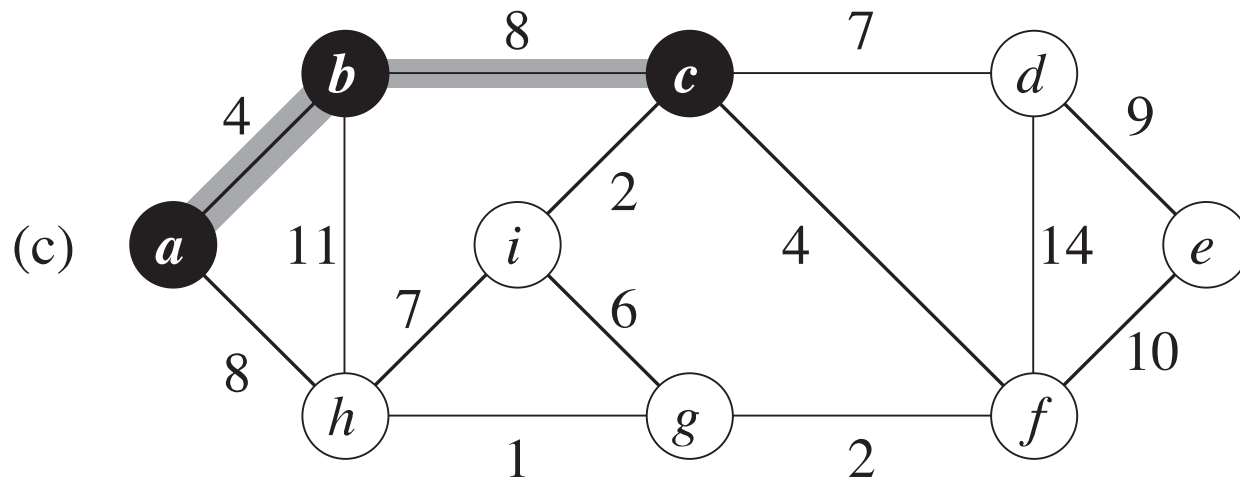
Prim( $V, E, r$ )  // returns an MST with root  $r$ 
for  $v \in V$ 
     $d[v] \leftarrow \infty$  and  $T[v] \leftarrow \text{NULL}$ 
 $Q \leftarrow V$ 
 $d[r] \leftarrow 0$   // root
while  $Q \neq \emptyset$ 
    Let  $u \leftarrow$  the vertex in  $Q$  with smallest  $d$ -value
     $Q \leftarrow Q - \{u\}$ 
    for each  $v \in V$  such that  $(u, v) \in E$ 
        if  $d[v] > w(u, v)$ 
             $d[v] \leftarrow w(u, v)$  and  $T[v] \leftarrow u$   // update!
return  $T$ 
    
```

Q : 현재 tree에 포함되지 않은 vertex들
 $d[v]$: v 를 현재 MST에 연결하기 위한 cost
 $T[v]$: 현재 tree에서 vertex v 의 부모

Prim's Algorithm

$$Q = \{ \quad d, e, f, g, h, i \}$$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>
<i>d</i>	0	4	8	7	∞	4	∞	8	2
<i>T</i>		<i>a</i>	<i>b</i>	<i>c</i>		<i>c</i>		<i>a</i>	<i>c</i>



```

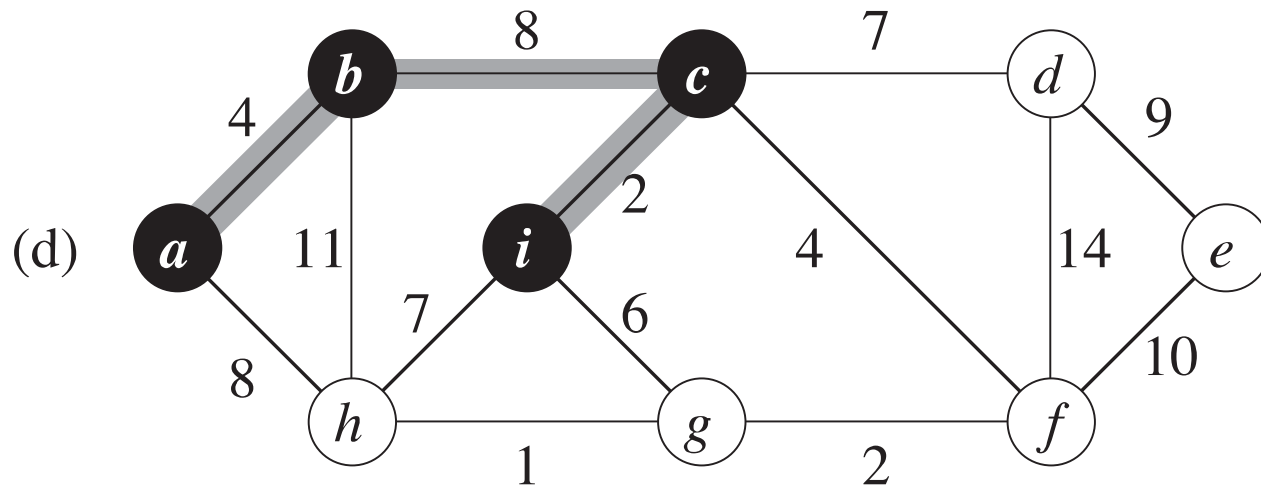
Prim( $V, E, r$ )  // returns an MST with root  $r$ 
for  $v \in V$ 
     $d[v] \leftarrow \infty$  and  $T[v] \leftarrow \text{NULL}$ 
 $Q \leftarrow V$ 
 $d[r] \leftarrow 0$   // root
while  $Q \neq \emptyset$ 
    Let  $u \leftarrow$  the vertex in  $Q$  with smallest  $d$ -value
     $Q \leftarrow Q - \{u\}$ 
    for each  $v \in V$  such that  $(u, v) \in E$ 
        if  $d[v] > w(u, v)$ 
             $d[v] \leftarrow w(u, v)$  and  $T[v] \leftarrow u$   // update!
return  $T$ 
    
```

Q : 현재 tree에 포함되지 않은 vertex들
 $d[v]$: v 를 현재 MST에 연결하기 위한 cost
 $T[v]$: 현재 tree에서 vertex v 의 부모

Prim's Algorithm

$$Q = \{ \quad d, e, f, g, h \quad \}$$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>
<i>d</i>	0	4	8	7	∞	4	6	7	2
<i>T</i>		<i>a</i>	<i>b</i>	<i>c</i>		<i>c</i>	<i>i</i>	<i>i</i>	<i>c</i>



```

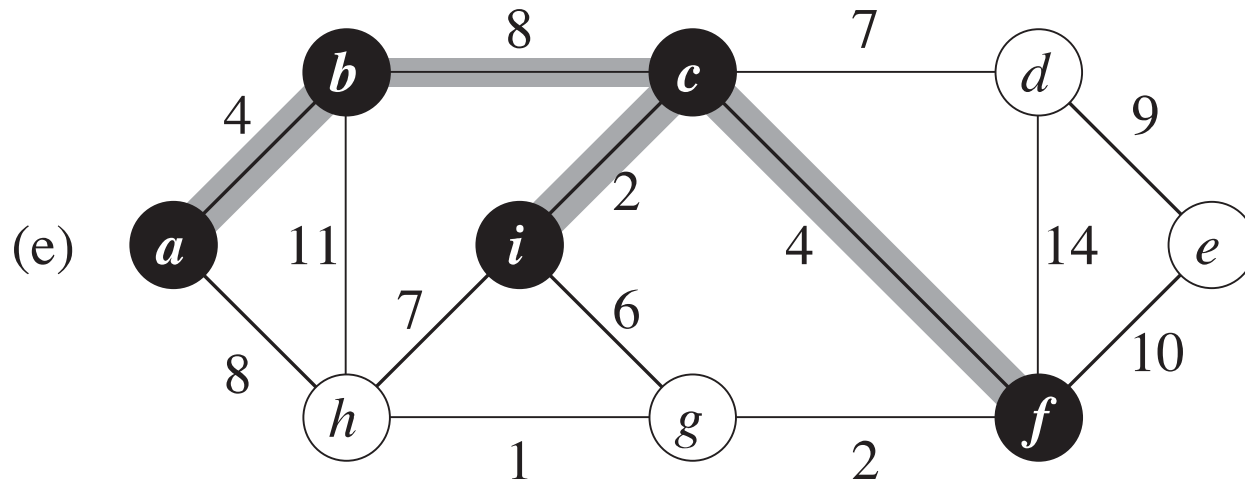
Prim(V, E, r)  // returns an MST with root r
for v ∈ V
    d[v] ← ∞ and T[v] ← NULL
Q ← V
d[r] ← 0  // root
while Q ≠ ∅
    Let u ← the vertex in Q with smallest d-value
    Q ← Q − {u}
    for each v ∈ V such that (u, v) ∈ E
        if d[v] > w(u, v)
            d[v] ← w(u, v) and T[v] ← u  // update!
return T
    
```

Q: 현재 tree에 포함되지 않은 vertex들
d[*v*]: *v*를 현재 MST에 연결하기 위한 cost
T[*v*]: 현재 tree에서 vertex *v*의 부모

Prim's Algorithm

$$Q = \{ \quad d, e, \quad g, h \quad \}$$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>
<i>d</i>	0	4	8	7	10	4	2	7	2
<i>T</i>		<i>a</i>	<i>b</i>	<i>c</i>	<i>f</i>	<i>c</i>	<i>f</i>	<i>i</i>	<i>c</i>



```

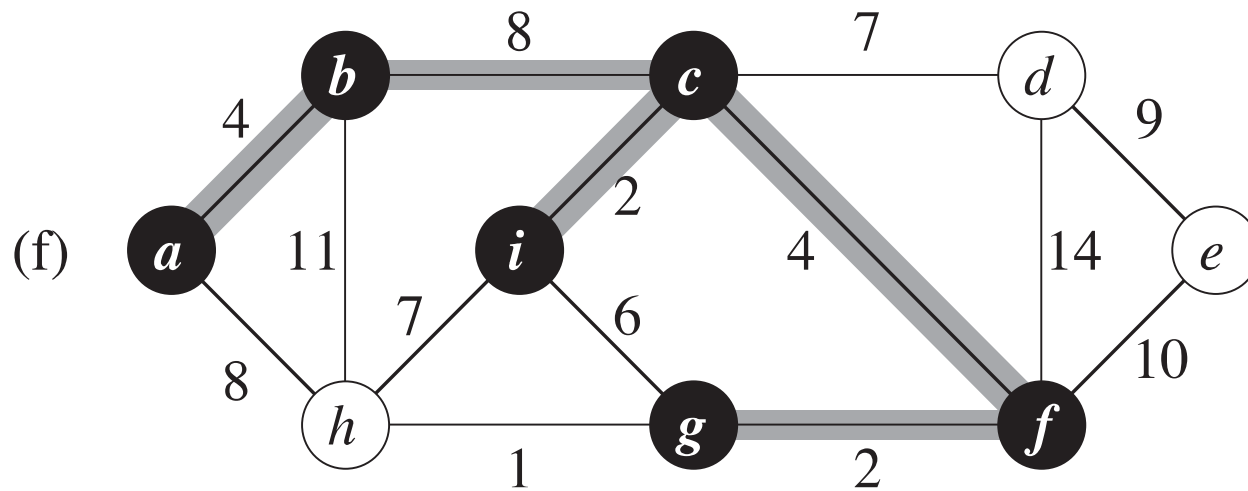
Prim(V, E, r)  // returns an MST with root r
for v ∈ V
    d[v] ← ∞ and T[v] ← NULL
Q ← V
d[r] ← 0  // root
while Q ≠ ∅
    Let u ← the vertex in Q with smallest d-value
    Q ← Q − {u}
    for each v ∈ V such that (u, v) ∈ E
        if d[v] > w(u, v)
            d[v] ← w(u, v) and T[v] ← u  // update!
return T
    
```

Q: 현재 tree에 포함되지 않은 vertex들
d[*v*]: *v*를 현재 MST에 연결하기 위한 cost
T[*v*]: 현재 tree에서 vertex *v*의 부모

Prim's Algorithm

$Q = \{ \quad d, e, \quad h \}$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>
<i>d</i>	0	4	8	7	10	4	2	1	2
<i>T</i>		<i>a</i>	<i>b</i>	<i>c</i>	<i>f</i>	<i>c</i>	<i>f</i>	<i>g</i>	<i>c</i>



```

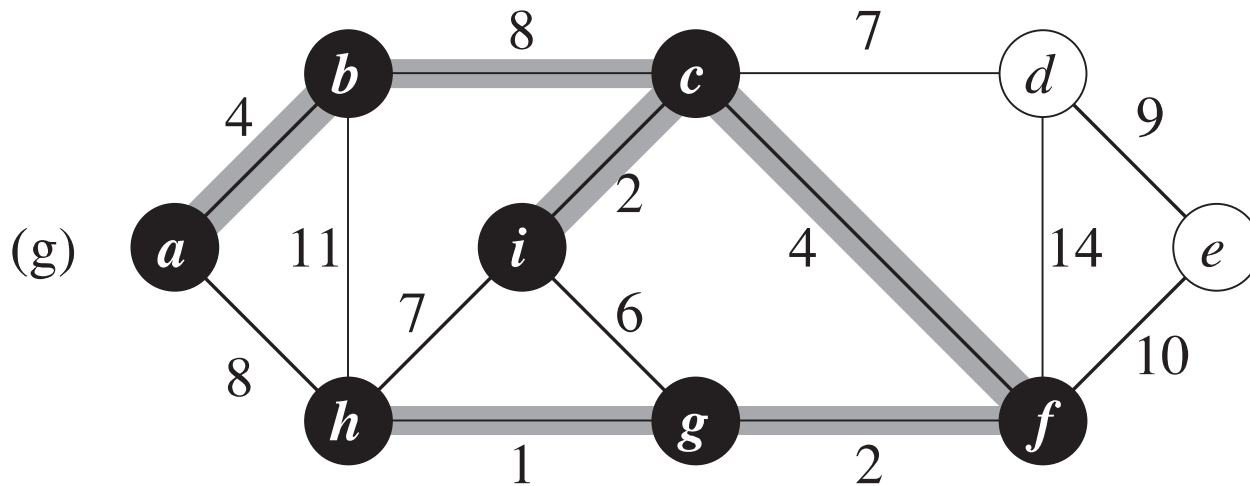
Prim( $V, E, r$ )  // returns an MST with root  $r$ 
for  $v \in V$ 
     $d[v] \leftarrow \infty$  and  $T[v] \leftarrow \text{NULL}$ 
 $Q \leftarrow V$ 
 $d[r] \leftarrow 0$   // root
while  $Q \neq \emptyset$ 
    Let  $u \leftarrow$  the vertex in  $Q$  with smallest  $d$ -value
     $Q \leftarrow Q - \{u\}$ 
    for each  $v \in V$  such that  $(u, v) \in E$ 
        if  $d[v] > w(u, v)$ 
             $d[v] \leftarrow w(u, v)$  and  $T[v] \leftarrow u$   // update!
return  $T$ 
    
```

Q : 현재 tree에 포함되지 않은 vertex들
 $d[v]$: v 를 현재 MST에 연결하기 위한 cost
 $T[v]$: 현재 tree에서 vertex v 의 부모

Prim's Algorithm

$Q = \{ \quad \quad \quad d, e \quad \quad \}$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>
<i>d</i>	0	4	8	7	10	4	2	1	2
<i>T</i>		<i>a</i>	<i>b</i>	<i>c</i>	<i>f</i>	<i>c</i>	<i>f</i>	<i>g</i>	<i>c</i>



```

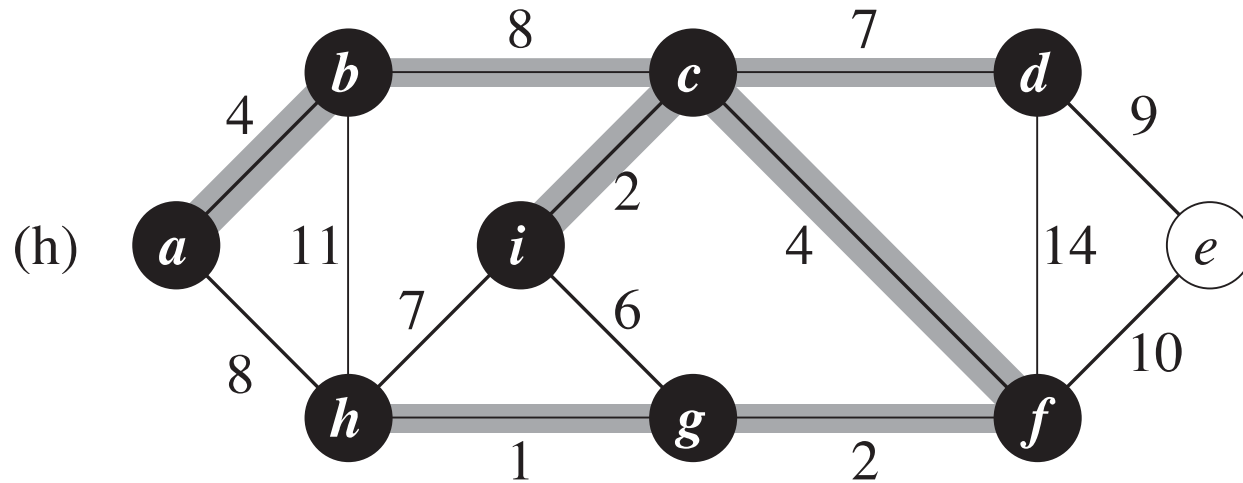
Prim( $V, E, r$ )  // returns an MST with root  $r$ 
for  $v \in V$ 
     $d[v] \leftarrow \infty$  and  $T[v] \leftarrow \text{NULL}$ 
 $Q \leftarrow V$ 
 $d[r] \leftarrow 0$   // root
while  $Q \neq \emptyset$ 
    Let  $u \leftarrow$  the vertex in  $Q$  with smallest  $d$ -value
     $Q \leftarrow Q - \{u\}$ 
    for each  $v \in V$  such that  $(u, v) \in E$ 
        if  $d[v] > w(u, v)$ 
             $d[v] \leftarrow w(u, v)$  and  $T[v] \leftarrow u$   // update!
return  $T$ 
    
```

Q : 현재 tree에 포함되지 않은 vertex들
 $d[v]$: v 를 현재 MST에 연결하기 위한 cost
 $T[v]$: 현재 tree에서 vertex v 의 부모

Prim's Algorithm

$Q = \{ \quad \quad \quad e \quad \quad \quad \}$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>
<i>d</i>	0	4	8	7	9	4	2	1	2
<i>T</i>		<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>c</i>	<i>f</i>	<i>g</i>	<i>c</i>



```

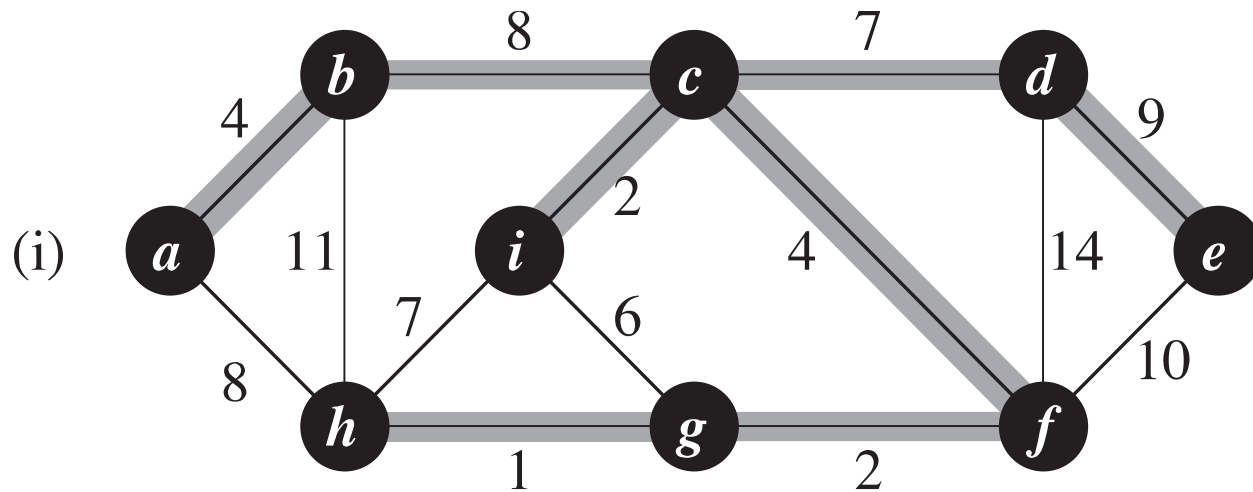
Prim( $V, E, r$ )  // returns an MST with root  $r$ 
for  $v \in V$ 
     $d[v] \leftarrow \infty$  and  $T[v] \leftarrow \text{NULL}$ 
 $Q \leftarrow V$ 
 $d[r] \leftarrow 0$   // root
while  $Q \neq \emptyset$ 
    Let  $u \leftarrow$  the vertex in  $Q$  with smallest  $d$ -value
     $Q \leftarrow Q - \{u\}$ 
    for each  $v \in V$  such that  $(u, v) \in E$ 
        if  $d[v] > w(u, v)$ 
             $d[v] \leftarrow w(u, v)$  and  $T[v] \leftarrow u$   // update!
return  $T$ 
    
```

Q : 현재 tree에 포함되지 않은 vertex들
 $d[v]$: v 를 현재 MST에 연결하기 위한 cost
 $T[v]$: 현재 tree에서 vertex v 의 부모

Prim's Algorithm

$Q = \{ \quad \}$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>	<i>i</i>
<i>d</i>	0	4	8	7	9	4	2	1	2
<i>T</i>		<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>c</i>	<i>f</i>	<i>g</i>	<i>c</i>



```

Prim(V, E, r)  // returns an MST with root r
for v ∈ V
    d[v] ← ∞ and T[v] ← NULL
Q ← V
d[r] ← 0  // root
while Q ≠ ∅
    Let u ← the vertex in Q with smallest d-value
    Q ← Q − {u}
    for each v ∈ V such that (u, v) ∈ E
        if d[v] > w(u, v)
            d[v] ← w(u, v) and T[v] ← u  // update!
return T
    
```

Q: 현재 tree에 포함되지 않은 vertex들
d[*v*]: *v*를 현재 MST에 연결하기 위한 cost
T[*v*]: 현재 tree에서 vertex *v*의 부모

Prim's Algorithm

- 집합 Q 에 필요한 연산
 - 초기화: $Q \leftarrow V$
 - ExtractMin
 - d 값이 가장 작은 걸 찾고 지우기
 - Update
 - d 값을 변경하기
- 시간복잡도 분석
 - 초기화 부분: $O(V) + (Q \leftarrow V \text{ 연산에 드는 시간})$
 - 메인루프(while)
 - ExtractMin 연산을 $|V|$ 번 수행
 - Update 연산을 최대 $|E|$ 번
 - 집합 Q 를 어떤 자료구조로 구현하느냐에 따라 달라진다

```
Prim( $V, E, r$ ) // returns an MST with root  $r$ 
for  $v \in V$ 
     $d[v] \leftarrow \infty$  and  $T[v] \leftarrow NULL$ 
 $Q \leftarrow V$ 
 $d[r] \leftarrow 0$  // root
while  $Q \neq \emptyset$ 
    Let  $u \leftarrow$  the vertex in  $Q$  with smallest  $d$ -value
     $Q \leftarrow Q - \{u\}$ 
    for each  $v \in V$  such that  $(u, v) \in E$ 
        if  $d[v] > w(u, v)$ 
             $d[v] \leftarrow w(u, v)$  and  $T[v] \leftarrow u$  // update!
return  $T$ 
```

Prim's Algorithm

- 시간복잡도 분석
 - 초기화 부분: $O(V) + (Q \leftarrow V \text{ 연산에 드는 시간})$
 - 메인루프(while)
 - ExtractMin 연산을 $|V|$ 번 수행
 - Update 연산을 최대 $|E|$ 번
- 집합을 표현할 수 있는 자료구조
 - Array, list 등 각종 container 구조
 - Q 를 Array로 구현할 경우
 - 초기화 $O(V)$, ExtractMin $O(V)$, Update $O(1) \rightarrow$ 총 $O(V^2+E) = O(V^2)$ 시간
 - ExtractMin과 Update 등을 효율적으로 처리하는 자료구조는?
 - Min-HEAP!

```
Prim( $V, E, r$ ) // returns an MST with root  $r$ 
for  $v \in V$ 
     $d[v] \leftarrow \infty$  and  $T[v] \leftarrow NULL$ 
 $Q \leftarrow V$ 
 $d[r] \leftarrow 0$  // root
while  $Q \neq \emptyset$ 
    Let  $u \leftarrow$  the vertex in  $Q$  with smallest  $d$ -value
     $Q \leftarrow Q - \{u\}$ 
    for each  $v \in V$  such that  $(u, v) \in E$ 
        if  $d[v] > w(u, v)$ 
             $d[v] \leftarrow w(u, v)$  and  $T[v] \leftarrow u$  // update!
return  $T$ 
```


Prim's Algorithm

- 시간복잡도 분석
 - 초기화 부분: $O(V) + (Q \leftarrow V \text{ 연산에 드는 시간})$
 - 메인루프(while)
 - ExtractMin 연산을 $|V|$ 번 수행
 - Update 연산을 최대 $|E|$ 번
- Binary Heap (이진트리를 이용한 Heap구조)
 - Insert: $O(\log N)$
 - ExtractMin: $O(\log N)$
 - Update: $O(\log N)$
 - 총 시간 복잡도: $O(V \log V + V \log V + E \log V) = O(E \log V)$
- Heap 자료구조도 여러가지가 있다
 - Fibonacci Heap을 사용하는 경우 Update가 $O(1)$ 시간에 가능
 - 총 시간 복잡도: $O(V \log V + V \log V + E) = O(E + V \log V)$

```
Prim(V, E, r) // returns an MST with root r
for v ∈ V
    d[v] ← ∞ and T[v] ← NULL
Q ← V
d[r] ← 0 // root
while Q ≠ ∅
    Let u ← the vertex in Q with smallest d-value
    Q ← Q - {u}
    for each v ∈ V such that (u, v) ∈ E
        if d[v] > w(u, v)
            d[v] ← w(u, v) and T[v] ← u // update!
return T
```

Prim's Algorithm

- Prim's Algorithm의 시간복잡도
 - Adjacency Matrix + Array
 - $O(V^2)$
 - Adjacency List + Binary Heap
 - $O(E \log V)$
 - Adjacency List + Fibonacci Heap
 - $O(E + V \log V)$

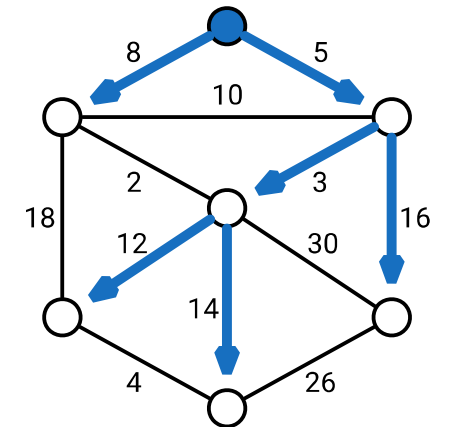
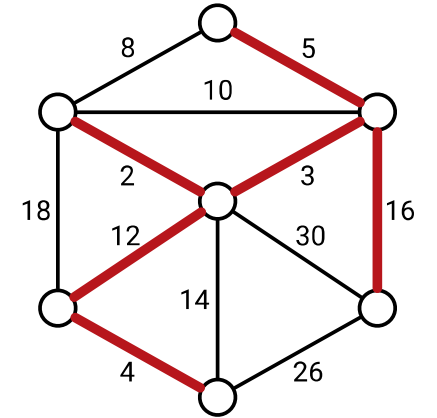
```
Prim(V, E, r) // returns an MST with root r
Initialize Q as a min-heap structure
for v ∈ V
    d[v] ← ∞ and T[v] ← NULL
    Q.Insert(v, d[v]) // key is d-value
d[r] ← 0 and Q.Update(r, d[r]) // root
while Q ≠ ∅
    u ← Q.ExtractMin() // the vertex in Q with smallest d-value
    for each v ∈ V such that (u, v) ∈ E
        if d[v] > w(u, v)
            d[v] ← w(u, v) and T[v] ← u
            Q.Update(v, d[v]) // Update the key value for v
return T
```

Shortest Paths

- Let $G = (V, E)$ be a connected graph with non-negative weight
 - Weight: $w(u, v) \geq 0$ for each edge $(u, v) \in E$
- For any $s, t \in V$,
 - a shortest path from s to t is a path with smallest total weight (length)
 - The distance from s to t is the length of a shortest path from s to t
- Shortest path problem
 - Input: $G = (V, E)$ and $s, t \in V$
 - Output: a shortest path from s to t

Shortest Paths

- Many shortest path algorithms
 - Ford (1956), Dantzig (1957), Leyzorek et al. (1957), Minty (1958), Dijkstra (1959), Whiting and Hillier (1960)
 - Most actually solve a more general problem:
[Single-Source Shortest Path problem](#)
- [Single-Source Shortest Path](#) problem
 - Input: $G = (V, E)$ and $s \in V$
 - Output: shortest paths from s to every other vertex
- [Shortest path tree for \$s\$](#)
 - A spanning tree of G with root s
 - The union of shortest paths from s to every other vertex



Shortest Path Tree

- Shortest path tree for s
 - A spanning tree of G with root s
 - The union of shortest paths from s to every other vertex
- If there are multiple shortest paths, choose any one
 - The union of shortest paths from s forms a tree

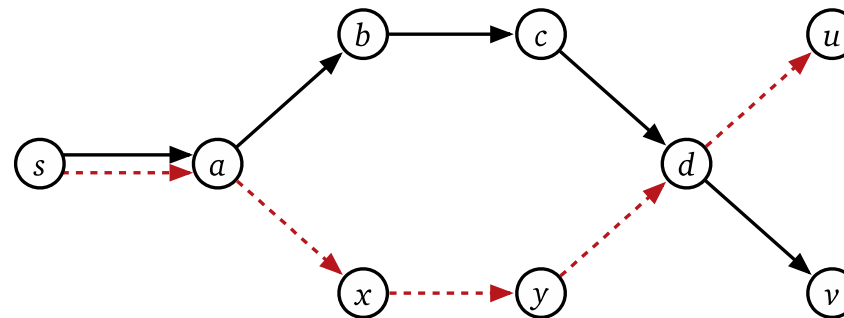
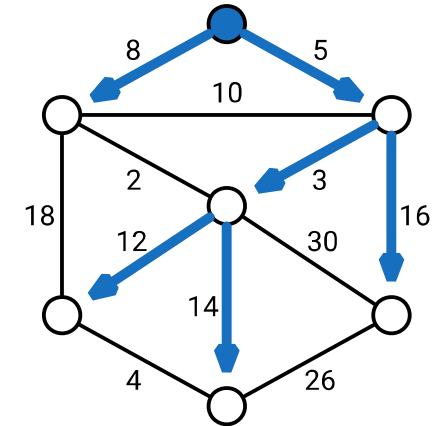


Figure 8.1. If $s \rightarrow a \rightarrow b \rightarrow c \rightarrow d \rightarrow v$ (solid) and $s \rightarrow a \rightarrow x \rightarrow y \rightarrow d \rightarrow u$ (dashed) are shortest paths, then $s \rightarrow a \rightarrow b \rightarrow c \rightarrow d \rightarrow u$ (along the top) is also a shortest path.

Dijkstra's Algorithm

- Single-Source Shortest Path problem
 - Input: $G = (V, E)$ and $s \in V$
 - Output: Shortest Path Tree(SPT) for s
- Dijkstra's Algorithm
 - Greedy algorithm!
 - Prim's Algorithm과 거의 똑같다
 - 주어진 vertex s 를 root로 시작하여 tree를 키워나가는 방식으로 SPT를 계산
 - 매 시점에서 현재 tree에 하나의 vertex와 edge를 추가
 - 다른 점은 다른 greedy choice 기준을 사용한다는 것
 - 추가되는 vertex는 현재 tree에 포함되지 않은 것 중
“s로부터 가장 가까운 것” → 즉 “s로부터의 distance가 가장 작은 것”을 선택

Dijkstra's Algorithm

- 배열 $d[v]$ 의 정의만 다르다
 - $d[v]$ 는 s 로부터 v 까지의 (현재까지의) distance (shortest path의 길이)

Prim(V, E, r) // returns an MST with root r

Initialize Q as a min-heap structure

for $v \in V$

$d[v] \leftarrow \infty$ and $T[v] \leftarrow NULL$

$Q.Insert(v, d[v])$ // key is d -value

$d[r] \leftarrow 0$ and $Q.Update(r, d[r])$ // root

while $Q \neq \emptyset$

$u \leftarrow Q.ExtractMin()$

for each $v \in V$ such that $(u, v) \in E$

if $d[v] > w(u, v)$

$d[v] \leftarrow w(u, v)$ and $T[v] \leftarrow u$

$Q.Update(v, d[v])$ // Update the key value for v

return T

Dijkstra(V, E, s) // returns an SPT for s

Initialize Q as a min-heap structure

for $v \in V$

$d[v] \leftarrow \infty$ and $T[v] \leftarrow NULL$

$Q.Insert(v, d[v])$ // key is d -value

$d[s] \leftarrow 0$ and $Q.Update(s, d[s])$ // root

while $Q \neq \emptyset$

$u \leftarrow Q.ExtractMin()$ // the vertex in Q with smallest d -value

for each $v \in V$ such that $(u, v) \in E$

if $d[v] > d[u] + w(u, v)$

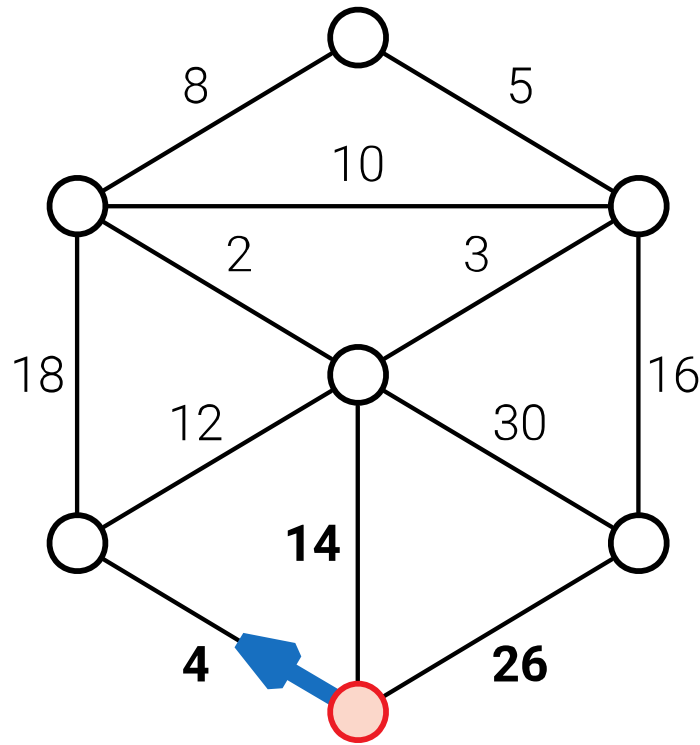
$d[v] \leftarrow d[u] + w(u, v)$ and $T[v] \leftarrow u$

$Q.Update(v, d[v])$ // Update the key value for v

return T

Dijkstra's Algorithm

$$Q = \{a, b, c, d, e, f, g\}$$



	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>
<i>d</i>							
<i>T</i>							

```
Dijkstra(V, E, s)  // returns an SPT for s
Initialize Q as a min-heap structure
for v ∈ V
    d[v] ← ∞ and T[v] ← NULL
    Q.Insert(v, d[v])    // key is d-value
d[s] ← 0 and Q.Update (s, d[s])  // root
while Q ≠ ∅
    u ← Q.ExtractMin()  // smallest d-value
    for each v ∈ V such that (u, v) ∈ E
        if d[v] > d[u] + w(u, v)
            d[v] ← d[u] + w(u, v) and T[v] ← u
            Q.Update (v, d[v])  // Update the key value for v
return T
```


Dijkstra's Algorithm

- Dijkstra's Algorithm의 시간복잡도
 - Adjacency Matrix + Array
 - $O(V^2)$
 - Adjacency List + Binary Heap
 - $O(E \log V)$
 - Adjacency List + Fibonacci Heap
 - $O(E + V \log V)$

All-Pairs Shortest Paths Problem

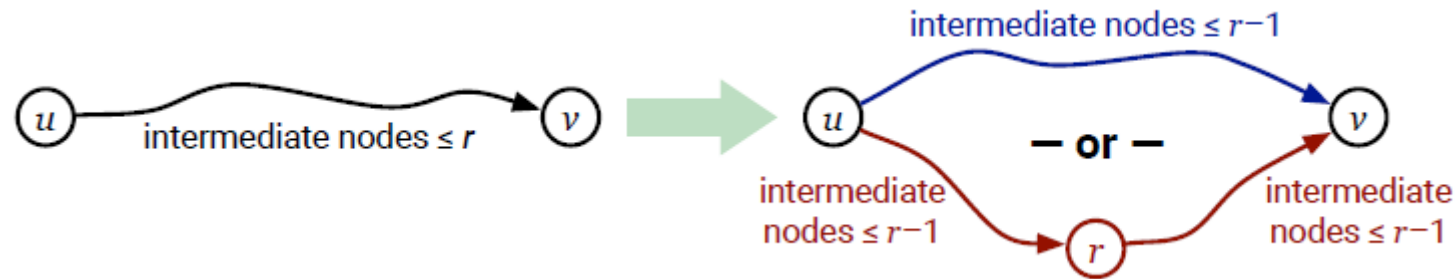
- All-Pairs Shortest Paths (APSP) problem
 - Input: $G = (V, E)$
Output: shortest paths from every vertex to every vertex
- APSP can be solved by solving SSSP $|V|$ times
 - Dijkstra's Algorithm $\times |V| \rightarrow O(EV \log V) = O(V^3 \log V)$ or $O(EV + V^2 \log V) = O(V^3)$ time
- APSP algorithms
 - Johnson's Algorithm
 - Shimbel (1954), Fischer and Meyer (1971)
 - Floyd-Warshall Algorithm $\rightarrow O(V^3)$
 - Floyd (1962), Ingberman (1962), Warshall (1962), Roy (1959), Kleene (1951)

Floyd-Warshall Algorithm

- All-Pairs Shortest Paths (APSP) problem
 - Input: $G = (V, E)$
Output: shortest paths from every vertex to every vertex
- Dynamic programming
 - Let $V = \{1, \dots, n\}$
 - For $u, v \in V$ and any integer r , define $\pi(u, v, r)$ to be the **shortest path** (if any) from u to v that **passes through only vertices $1, \dots, r$**
 - $\pi(u, v, n)$ is the true shortest path
 - Define $\text{dist}(u, v, r)$ to be the length of $\pi(u, v, r)$
 - Compute $\text{dist}(u, v, r)$ (and $\pi(u, v, r)$) for all vertices $1 \leq u, v, r \leq n$ in a bottom-up fashion using a nice recurrence relation

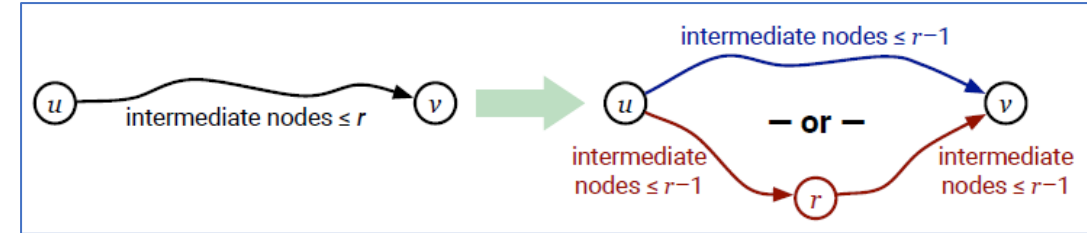
Floyd-Warshall Algorithm

- Recursive structure on $\pi(u, v, r)$



- Base case: when $r = 0$
 - $\pi(u, v, 0)$ is the edge (u, v) if $(u, v) \in E$, or it doesn't exist
- General case: when $r > 0$
 - Two cases: $\pi(u, v, r)$ passes through r or not
 - If $\pi(u, v, r)$ passes through $r \rightarrow \pi(u, v, r) = \pi(u, r, r-1) \cup \pi(r, v, r-1)$
 - If $\pi(u, v, r)$ does not pass through $r \rightarrow \pi(u, v, r) = \pi(u, v, r-1)$

Floyd-Warshall Algorithm



- Recursive structure on $\pi(u, v, r)$
 - Base case: when $r = 0$
 - $\pi(u, v, 0)$ is the edge (u, v) if $(u, v) \in E$, or it doesn't exist
 - General case: when $r > 0$
 - Two cases: $\pi(u, v, r)$ passes through r or not
 - If $\pi(u, v, r)$ passes through $r \rightarrow \pi(u, v, r) = \pi(u, r, r-1) \cup \pi(r, v, r-1)$
 - If $\pi(u, v, r)$ does not pass through $r \rightarrow \pi(u, v, r) = \pi(u, v, r-1)$
- $dist(u, v, r)$ 의 점화식
 - $dist(u, v, r) = \begin{cases} w(u, v) & \text{if } r = 0 \\ \min \left\{ \begin{array}{l} dist(u, v, r-1) \\ dist(u, r, r-1) + dist(r, v, r-1) \end{array} \right\} & \text{otherwise} \end{cases}$

Floyd-Warshall Algorithm

$$\text{dist}(u, v, r) = \begin{cases} w(u, v) & \text{if } r = 0 \\ \min \left\{ \begin{array}{l} \text{dist}(u, v, r-1) \\ \text{dist}(u, r, r-1) + \text{dist}(r, v, r-1) \end{array} \right\} & \text{otherwise} \end{cases}$$

```
FloydWarshall( $V, E$ )           // assume  $V = \{1, \dots, n\}$ 
  for  $u = 1$  to  $n$ 
    for  $v = 1$  to  $n$ 
       $\text{dist}[u, v, 0] \leftarrow w(u, v)$   //  $w(u, v) = \infty$  if  $(u, v) \notin E$ 
  for  $r = 1$  to  $n$ 
    for  $u = 1$  to  $n$ 
      for  $v = 1$  to  $n$ 
        if  $\text{dist}[u, v, r-1] < \text{dist}[u, r, r-1] + \text{dist}[r, v, r-1]$ 
           $\text{dist}[u, v, r] \leftarrow \text{dist}[u, v, r-1]$ 
        else
           $\text{dist}[u, v, r] \leftarrow \text{dist}[u, r, r-1] + \text{dist}[r, v, r-1]$ 
```

시간 $O(V^3)$, 공간 $O(V^3)$

Floyd-Warshall Algorithm

$$\text{dist}(u, v, r) = \begin{cases} w(u, v) & \text{if } r = 0 \\ \min \left\{ \begin{array}{l} \text{dist}(u, v, r-1) \\ \text{dist}(u, r, r-1) + \text{dist}(r, v, r-1) \end{array} \right\} & \text{otherwise} \end{cases}$$

```
FloydWarshall( $V, E$ )           // assume  $V = \{1, \dots, n\}$ 
  for  $u = 1$  to  $n$ 
    for  $v = 1$  to  $n$ 
       $\text{dist}[u, v] \leftarrow w(u, v)$  //  $w(u, v) = \infty$  if  $(u, v) \notin E$ 
  for  $r = 1$  to  $n$ 
    for  $u = 1$  to  $n$ 
      for  $v = 1$  to  $n$ 
        if  $\text{dist}[u, v] > \text{dist}[u, r] + \text{dist}[r, v]$ 
           $\text{dist}[u, v] \leftarrow \text{dist}[u, r] + \text{dist}[r, v]$ 
```

시간 $O(V^3)$, 공간 $O(V^2)$

마무리

- Graph
- Graph Algorithms
 - DFS, BFS, ...
- Minimum Spanning Tree
 - Kruskal's Algorithm
 - Prim's Algorithm
- Shortest Path
 - Dijkstra's Algorithm
 - Floyd-Warshall Algorithm

마무리

- Algorithm 분석
 - 시간복잡도 분석
 - 정확성 증명
- Algorithm 설계 방법론
 - Recursion, D&C,
 - Backtracking, DP
 - Greedy method
- 궁극의 목표는 더 효율적인 코드 만들기